Plectis: The Public Slice of a Private AI-Built System

The source code, the runnable fixtures, and what they do and do not show

Will Cook

July 2026

Abstract

For about ten months I have been building a private workflow and research system by directing AI coding agents. The private repository cannot be published: it holds personal records, credentials, live account state, and tooling that is not ready for release. Plectis is the part that can be published. It is a public repository containing real source code from that system, organised as 88 runnable components across seven areas, each with a small frozen input, a local command, and a written record of what happened when it ran. A stranger can clone it and run the pieces on their own machine, with no network access and no model calls, instead of taking a description of the private system on trust. This paper explains, from first principles, what is in the slice, what one run looks like, how the slice was cut from the private repository, and exactly what a passing check does and does not show.

1 What Plectis is

Plectis is the public slice of a private system: the source code and runnable fixtures that could be released, published as one repository at github.com/wcook04/plectis.

The private system behind it is a single working repository in which AI coding agents — programs such as Claude and Codex that read, write, and run code — build and maintain software under one person's direction. That repository stays private for ordinary reasons: it contains personal records, credentials, browser and account state, operator notes, and tools that are not finished. But much of what the agents built is ordinary, non-secret source code, and that part did not need to stay private. Plectis is that part, extracted and made standalone.

The consequence for a reader is simple. You do not have to take anyone's word about the private system, because the slice runs on your machine:

```
git clone https://github.com/wcook04/plectis && cd plectis
python3 -m pip install .
plectis tour --format text .
```

Everything is local. Plectis needs Python 3.11 or newer and nothing else: no third-party runtime dependencies, no network or model calls, and it never changes the source files it reads.

2 The private system behind it

To understand what the slice is a slice *of*, four plain facts about the private system are enough.

First, it is one repository, not a company or a product. Inside it, coding agents write software, prove mathematics in the Lean proof assistant, keep records of their own work, and operate

under written rules stored alongside the code. I am 22, an economics undergraduate, and I have built it alone in about ten months; the agents do the building and upkeep, and I set the direction and the rules.

Second, it is genuinely private. Beyond secrets, it contains a frontend, a video editor, recording tooling, and operating state that have never been released. No public artifact reconstructs it.

Third, a private system has a credibility problem. Any claim about it — "the agents keep auditable records," "the proof pipeline is real" — is unverifiable from outside. Describing it in more detail does not help; a description is still just prose.

Fourth, the response to that problem is to publish the checkable part rather than a longer description. Two public repositories exist for this purpose. Plectis shows the system's breadth: many mechanisms, each small and runnable. The companion Lean repository (Section 7) shows its depth: one sustained piece of machine-checked mathematics. Everything else remains private.

3 What is in the slice

The repository contains a small Python package (under `src/`), a corpus of 88 components, the frozen fixtures those components run on, the receipts their accepted runs wrote, tests, and generated documentation. The corpus is grouped into seven areas:

Area	Count	What its components do
Entry and reveal	2	Give a cold reader a first route and a guided public walkthrough.
Architecture and navigation	12	Index a project, discover patterns, route questions, and expose the standards the system is built on.
Formal mathematics and proof	20	Exercise the Lean proof-evidence pipeline: premise retrieval, tactic routing, verifier-trace repair, bounded witnesses, certificates.
Agent reliability and safety replays	20	Replay recorded agent failure modes: prompt injection, sandbox escape, memory poisoning, tool misuse, benchmark gaming.
Research and science replays	9	Replay scientific-workflow shapes: replication rubrics, forecasting reconciliation, materials-lab safety.
Import, projection, and drift	20	Bring non-secret source into the public tree and detect when generated pages drift from their sources.
Work, landing, and continuity	5	Record reversible work, dirty-tree landing decisions, and how interrupted runs resume.

The count describes the inventory, not quality; being in the list is not a score. What makes the corpus usable is that every component has the same shape. Each one states a single claim, names the command or replay that exercises it, ships the small frozen input it acts on, writes a receipt when it runs, and records a limit — one sentence saying what a passing result does *not* establish. A component that cannot state its claim, its command, and its limit is not accepted into the corpus.

Components enter the slice in two ways. Some are source modules copied byte-for-byte from the private repository, with fingerprints and attribution recorded, after a check that they contain nothing secret. Others are *replays*: where a private mechanism could not cross whole — because it touches live state or private data — the slice keeps the rule and a frozen case demonstrating

it, rather than the live machinery. Both kinds are labelled, so a reader always knows whether they are looking at working source or a bounded demonstration.

4 One run, end to end

On 17 July 2026, the standard first command was run against the Plectis checkout itself:

```
plectis tour --format text .
```

```
Plectis read 5044 project files and wrote a local record.  repo -> .microcosm

  Route taken     readme_onboarding_route  (one of 5 it found)
  Record written  .microcosm/  (68 local files, written beside your project)
  Your source     unchanged

  Every finding in that record carries three handles:
    Evidence   .microcosm/evidence/   (what backs the finding)
    Source     .microcosm/events.jsonl   (where the finding came from)
    Scope      does not authorize release, provider calls, whole-system
               correctness
```

In plain terms: the tool read the folder it was pointed at, chose one of five routes through it, and wrote a record of what it found in a separate local directory, leaving the folder itself untouched. File counts change with the checkout; the shape of the run does not. The record is ordinary files. You can open the route it chose, follow the evidence behind any finding, delete the whole record, run the command again, and compare.

From there, the useful review is not to read all 88 components. It is to take one claim through the whole chain:

1. Pick a component from the generated map (`ORGANS.md`, or the component browser on the website).
2. Read its claim and its limit.
3. Run its named command or replay on its shipped fixture.
4. Compare what it wrote with the tracked receipt, rather than with the prose around it.
5. Ask `plectis comprehend --first-action "<claim>" --format text` to route any other question to the component that owns it.

For the whole-repository floor, `make ci` runs the same install, test, and smoke path the public continuous-integration run uses, and `./bootstrap.sh` probes a cold clone before installing anything.

5 How the slice was cut

Cutting a public slice out of a private repository is mostly a question of discipline at the boundary, and the discipline is itself part of the public code rather than a policy document.

Imports are checked. A source module crosses only after an import step that verifies it is non-secret, records its fingerprint and origin, and logs what was omitted or refused. The copied bodies in the public tree are checked against those records, so “this is the real source” is a testable statement rather than an assurance.

Projections are generated. The component map, the architecture page, and the website are built from the repository's machine-readable component records, not maintained by hand, so the list of parts cannot silently drift from the parts that exist. When a generated page and the source disagree, the source wins; one component family exists specifically to detect such drift.

The boundary is explicit. The public tree does not read private files at runtime, does not contain them, and is a cross-section rather than a reconstruction: nothing here claims to be the whole system, and no amount of inspection of Plectis will reveal the private state that stayed behind.

One historical note belongs here. The project was published under the name Microcosm until June 2026, when it was renamed Plectis to avoid confusion with the earlier Southampton Microcosm hypermedia system. The old name survives only where compatibility requires it, such as the `microcosm_core` import path and the `.microcosm/` record directory.

6 What a passing check shows — and what it does not

A passing component run shows something narrow and real: this mechanism, on this recorded input, produced this result, on your machine, just now. That is the whole point of the slice — narrow facts a stranger can establish without trusting anyone.

It is equally important to say what no amount of green output establishes.

- **No release or production authority.** Passing checks do not decide that software should be published, deployed, or relied on.
- **No domain authority.** The safety replays are recorded demonstrations, not security certification; the proof components feed a pipeline, they are not the Lean kernel; nothing here is professional advice.
- **No whole-system claim.** The slice passing does not show the private system is correct, and the frozen fixtures that make publication safe are weak evidence about behaviour in uncontrolled conditions.
- **No independent evaluation.** Plectis is one project maintained under one direction. It has not been compared against other systems, and nothing here should be read as a measured performance result.

These limits are stored as machine state on each component and checked by tests, because a disclaimer at the bottom of a page is easy to lose.

7 The companion Lean repository

Breadth without depth would be easy to fake with many shallow pieces, so the second public repository, plectis-lean-erdos249-257, publishes one deep artifact: a Lean 4 development built around two open Erdős problems in number theory, #249 and #257. Its recorded public snapshot contains 633 Lean modules and 11,467 theorem-like declarations, every one checked by a pinned Lean kernel; those are scale counts, not mathematical claims. The repository is precise about the line between what is proved and what is not: it does not solve either problem in full, and the exact propositions that remain open are named rather than glossed. The public Plectis checkout holds that work to the same floor — its `make check` rejects proof placeholders, project-defined axioms, and unsafe declarations in every Lean fixture it ships.

The two repositories are the same demonstration at different depths: the private system built both, and both are published in the only form that counts for a stranger — source you can read and checks you can run.

What to do with it

Clone the repository, run `plectis tour --format text .`, and open the record it writes. Then pick one component whose claim you doubt, run it, and read its receipt and its limit. The website at wcook04.github.io/plectis carries the same corpus as a browsable map, and `CONTRIBUTING.md` explains how to report anything that does not hold up. The repository is offered for inspection, experimentation, and learning — and it earns attention exactly to the extent that its pieces run as described.